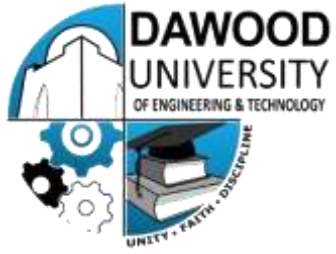

Programming Fundamentals

By

Dr Suhail Asghar

Dawood University Of Engineering & Technology Karachi

Object-Oriented Programming (OOP)

Why Object-Oriented Programming?

- Motivation: Procedural programming organizes code around *functions*. OOP organizes code around *objects* that combine:
  - Data (attributes)
  - Behavior (methods)
- Benefits
  - Modularity
  - Reusability
  - Maintainability
  - Scalability
  - Better mapping to real-world systems
- Real-World Analogy
- A Student is not just data (name, roll no) or behavior (study, attend). It is a *unit* containing both.

Core OOP Concepts

- Class – Blueprint
- Object – Instance of a class

1. Encapsulation: Data hiding
2. Abstraction: Essential features only
3. Inheritance: Reuse & extension
4. Polymorphism: One interface, many behaviors

Classes and Objects in C++

- Class Definition

```
class Student {
public:
int rollNo;
string name;

void display() {
cout << rollNo << " " <<
name << endl;
}    };
```

- Object Creation

```
Student s1;
s1.rollNo = 101;
s1.name = "Ali";
s1.display();
```

Encapsulation & Access Specifiers

- **Access Levels**
- Public: accessible everywhere
- Private“: accessible only inside class
- Protected: for inheritance

Encapsulation

- Wrapping data and functions together AND controlling access to that data.
- Access Levels
  - Public: accessible everywhere
  - Private: accessible only inside class
  - Protected: for inheritance

Example

```
class Student {
private:
    int marks;
public:
    void setMarks(int m)
{
        if (m >= 0 && m
<= 100)
            marks = m;
}
    int getMarks() {
        return marks;
    }    };
```

Inheritance

- **Purpose**
  - Code reuse
  - Logical hierarchy
- **Types of Inheritance**
  - Single
  - Multilevel
  - Hierarchical
  - Multiple (with caution)

Syntax

```
class Person {
public:
string name;
};

class Student : public Person {
public:
int rollNo;
};
```

Write a C++ program to demonstrate single inheritance using a base class Animal and a derived class Dog. The base class should contain a data member name and a member function eat(). The derived class should contain a member function bark().

```
#include <iostream>
using namespace std;
class Animal {
protected:
    string name;   // accessible in derived class

public:
    void setName(string n) {
        name = n;    }
    void eat() {
        cout << name << " is eating" << endl;
    }  };

class Dog : public Animal {   // public inheritance
public:
    void bark() {
        cout << name << " is barking" << endl;
    }  };

int main() {
    Dog d;
    d.setName("Buddy");  // inherited function
    d.eat();             // inherited function
    d.bark();            // own function
    return 0;
}
```

Abstraction

- Abstraction is the process of exposing only the essential features of an object while hiding the implementation details.
- It answers the question: what an object does, not how it does it.

- Purpose
  - Reduce complexity
  - Improve maintainability
  - Enforce a clear contract between interface and implementation
- How Abstraction Is Achieved in C++
  1. Abstract classes
  2. Pure virtual functions
  3. Interfaces (via abstract classes)

Example (Run-Time Abstraction)

```
#include <iostream>
using namespace std;

class Shape { // Abstract class
public:
    virtual void draw() = 0;   // Pure virtual function     };

class Circle : public Shape {     // Derived class
public:
    void draw() {
        cout << "Drawing Circle" << endl;
    }  };
```

```
int main() {
    Shape* s;
    Circle c;
    s = &c;
    s->draw();
    return 0;
}
```

Polymorphism

- Polymorphism means one interface, multiple forms. It allows the same function call to behave differently depending on the object type.

- Types of Polymorphism in C++
  1. Compile-Time Polymorphism
     - Function overloading
     - Operator overloading
  2. Run-Time Polymorphism
     - Function overriding
     - Achieved using virtual functions

Example (Run-Time Polymorphism)

```
#include <iostream>
using namespace std;
class Animal {
public:
    virtual void sound() {
        cout << "Animal makes a sound" << endl;
    } };

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks" << endl;    } };
```

```
int main() {
    Animal* a;
    Dog d;
    a = &d;
    a->sound();   // Calls Dog's sound()
    return 0;
}
```

- Base-class pointer refers to derived-class object.
- Correct function is chosen at runtime.
- Requires virtual keyword.